

Comparative performance evaluation of island particle swarm algorithm applied to solve constrained and unconstrained optimization problems

Houda Abadlia*, Nadia Smairi and Khaled Ghedira

LARIA Laboratory, Manouba National School of Computer Science, University of Manouba, Tunisia

Abstract. Distributed evolutionary computation has been efficiently used, in last decades, to solve complex optimization problems. Island model (IM) is considered as a distributed population paradigm employed by evolutionary algorithms to preserve the diversification and, thus, to improve the local search. In this article, we study different island model techniques integrated in to particle swarm optimization (PSO) algorithm in order to overcome its drawbacks: premature convergence and lack of diversity. The first IMPSO approach consists in using the migration process in a static way to enhance the police migration strategy. On the other hand, the second approach, called dynamic-IMPSON, consists in integrating a learning strategy in case of migration. The last version called constrained-IMPSON utilizes a stochastic technique to ensure good communication between the sub-swarms. To evaluate and verify the effectiveness of the proposed algorithms, several standard constrained and unconstrained benchmark functions are used. The obtained results confirm that these algorithms are more efficient in solving low-dimensional problems (CEC'05), large-scale optimization problems (CEC'13) and constrained problems (CEC'06), compared to other well-known evolutionary algorithms.

Keywords: Particle swarm optimization, Island model, diversity, constrained optimization, unconstrained optimization

1. Introduction

Metaheuristics have been, recently, applied in different fields such as engineering problems, operation research, etc. Some of them imitate the swarm intelligence behavior based on groups of interacting agents. The optimization procedure relies on a collaborative research behavior. For example, PSO algorithm simulates a group of fishes and birds [1]. However, ant colony optimization algorithm (ACO) was inspired from the nature communication between ants to find

the shortest path from their origin to a food source [2]. The artificial bee colony (ABC) algorithm mimics the foraging behavior of honeybees in seeking quality food [3]. On the other hand, cuckoo search algorithm (CS) was inspired from the breeding behavior of certain species of cuckoos. Each solution is called nest or cuckoo [4]. Dragon fly algorithm (DA) simulates the static and dynamic swarming behaviors of certain species set. The static and dynamic ways represent the exploitation and exploration strategies of DA. For the exploitation strategy, a set of dragonflies make the swarms migrate in one direction and to distract from enemies. For the exploration strategy, dragonflies fly black in small groups, search for food and attract flying preys [5].

*Corresponding author. Houda Abadlia, LARIA Laboratory, Manouba National School of Computer Science, University of Manouba, Tunisia. E-mail: houdaabadlia@gmail.com.

Indeed, the island model is a distributed way of implementing evolutionary algorithms (EAs). It divides a single population into semi-isolated sub-populations connected by the migration process. It is based on the following parameters: the number of islands, migration size (the number of individuals that migrate), migration interval (number of generations between migrations) and migration topology which specifies the connection between islands. Implementation decisions about the selection of the migrants and their replacement must also be defined.

As reported in the literature, several EAs utilized island model parameters to improve their efficiency when applied in diversification strategy. A study of the migration topologies for differential evolution and simulated learning showed the feasible paths of sending and receiving the individuals among islands [6]. Dynamic island model strategies were applied by Candan et al. to enhance the search performance of the genetic algorithm [7]. Indeed, island based on search harmony uses the random ring topology and the best-worst policy migration to maintain diversity in the search space [8]. PSO algorithm was inspired from the social interaction behaviours of bird flocking and fish schooling [1]. Because of its easy implementation and wide application to solve optimization problems, such as continuous, discrete and engineering problems, PSO has been widely applied in many research works. However, it has the ability to quickly converge, which can lead to stagnation at local optima [9, 10]. Indeed, a hybrid model can be efficiently used to find a compromise between the advantages and disadvantages of several optimization methods. Thus, in this work, we propose a framework of island-PSO algorithms utilized to solve constrained and unconstrained mono-objective optimization problems:

- The first approach is integrated statistically the IM techniques. The migration process is triggered after certain number of iterations. Different types of communication topologies among the sub-swarms with various numbers of islands are used to explore the search space. Besides, the migration policy is applied to replace the solutions between islands using various elitist strategies.
- The second version uses dynamically the IM parameters, especially the migration policy and migration size, according to the evolution process of each island based on a learning strategy. It consists in tuning the IM parameters to improve PSO performance. The effect of the number of

islands on the swarm size is also studied in this paper.

- The third approach, named CIMPSO, is generally employed to solve constrained optimization problems. In CIMPSO, two constrained handling mechanisms are integrated into the IMSPO algorithm. The communication strategy of Distributed Stochastic Algorithm (DSA) used to exchange the best solution among the islands. It is performed at the migration moment according to violation constraints in order to maintain diversity in the swarm.

The remainder of this paper is structured as follows. Section 2 reviews some of the related works of PSO algorithm. Section 3 introduces the proposed algorithms. Section 4 reports the experimental results and discussions. Finally, section 5 provides some concluding remarks.

2. Related works

2.1. Mono-objective optimization

The optimization problem can be formulated by applying an objective function to be optimized and a set of equality and inequality constraints to be met simultaneously:

$$\begin{aligned} & \text{Minimize } f(x) \\ & \text{Subject to } g(x) \leq 0 \text{ (} m \text{ inequality constrained)} \\ & \quad h(x) = 0 \text{ (} p \text{ equality constrained)} \\ & \quad \text{and } x \in R^n, g(x) \in R^m, h(x) \in R^p \end{aligned}$$

where $f(x)$ is the objective function and $g(x)$ and $h(x)$ vectors represent respectively m inequality constraints and p denotes equality constraints, respectively.

2.2. Particle swarm optimization

PSO considers particles population that flies through the problem hyperspace with given velocities. At each iteration, the velocities of the particles are stochastically adjusted according to the influence of their best knowledge (pbest) and the best knowledge of their neighbors (gbest). Then, a new position to be evaluated is computed. In fact, particle displacement is affected by three components:

- Physical component which keeps the current direction of displacement;
- Cognitive component which moves towards the best explored region; and
- Social component in which the particle relies on the experience of its congeners and, then, it moves towards the best region already explored by its neighbors.

The displacement of each particle in the search space is based on its current position and the update of its velocity. Let $x_i(t)$ be the position of the particle p_i at the time step t . This position is modified by the addition of the velocity $v_i(t)$ of the current position:

$$x_i(t) = x_i(t-1) + v_i(t) \quad (1)$$

Each particle in the swarm changes its velocity according to two types of essential information. The first information, called *pbest*, shows its personal experience. It represents the best position found by the particle during the search process. However, the second information is about the best position found by the whole swarm. It is obtained by knowing how the other particles perform their searches. The principle change of the velocity is defined as follows:

$$v_i(t) = w.v_i(t-1) + r_1.c_1.(x_i(pbest_i) - x_i(t)) + r_2.c_2.(x_gbest - x_i(t)) \quad (2)$$

The $v_i(t)$ is the velocity of the i th particle ($i \in 1, 2, \dots, s$) of the j th dimension where c_1, c_2 called acceleration coefficients, are the learning factors that will be fixed throughout the whole process; r_1, r_2 are two random numbers in the range $[0,1]$ selected uniformly for each dimension at each iteration, $v_i(t-1)$ is the physical component; $r_1.c_1.(x_i(pbest_i) - x_i(t))$ corresponds to the cognitive component where c_1 controls the cognitive behavior of the particle; and $r_2.c_2.(x_gbest - x_i(t))$ designates the social component where c_2 controls the social behavior of the particle.

2.3. The proposed variants of PSO

The PSO algorithm was widely applied to solve optimization problems [11] because of its easy implementation and efficiency in solving different test functions and engineering problems. It is also intensively used to deal with real-world optimization problems for resource allocation [12], image segmentation [13], mechanical engineering [14] and many other tasks [15], [16]. Although PSO performance has been significantly improved in recent years, it has

some limitations. Besides, like other EAs, this algorithm suffers from premature convergence because of the lack of diversity. To improve the search performance, many PSO variants have been introduced in the last decades. They can be summarized as follows: The cooperative swarm optimization algorithm [17] is based on a parallel model in which the swarm is divided into sub-swarms. Two versions were presented: the cooperative version that uses the master swarm (evolved based on its knowledge and the knowledge of the slave particles) and the slave swarms performing the PSO operations. The collaborative version including a master swarm that improves the characteristics of particles relying on an antagonistic scenario or a synergistic scenario.

To enhance PSO performance, artificial bee colony PSO was proposed [18]. The global best solutions obtained by PSO and by ABC were combined to obtain the best results. On the other hand, the obtained solution was given to the populations of PSO and ABC as the global best and neighbor food source for onlooker bees, respectively. Information flow between particle swarm and bee colony allows increasing the global and local search abilities of the hybrid approach. Another PSO algorithm, called AugPSO [19], adopts two strategies: boundary shifting and particle position resetting to optimize the lattice structures. In fact, the boundary-shifting technique forces the displacement of particles towards the boundary of feasible and unfeasible regions to increase the convergence rate in the search space. However, the technique of particle-position-resetting was motivated by the mutation schema inspired by the genetic algorithms to improve the diversification strategy.

Another version of an adaptive PSO was developed by [20] to optimize the hyper-parameter of long-term and short-term memory networks (LSTM). PSO is hybridized with opposition based on learning algorithm to select and adjust the optimal weights and the learning rates for the deep learning algorithm with the minimum number of iterations. Besides, a PSO framework was proposed by [21]. The developed strategy consists in exchanging the random initialization of the swarm by homotopy analysis technique that uses the information provided by the solved optimization problem to seal with an unknown problem. To solve feature selection problem, the authors proposed three layers of particles. In each layer, the particles learn from a random learning exemplar selection strategy to maintain the diversity. The PSO algorithm was also combined with a Gaussian

distribution technique to enhance the exploitation in the search space [22]. Likewise, PSO algorithm was utilized in Deep Auto-encoder algorithm (DAE), instead of the gradient descent algorithm, to improve the convergence rate. PSO trained DAE to extract the latent representation and the degeneracy method used to avoid the premature convergence and to minimize the execution time [23].

To attain this objective, the master-slave model [24] was employed in a distributed strategy in which the population was divided into one master swarm and several slave swarms. The slave swarms executed independently a single PSO step to maintain the diversity of the particles, while the master swarm evolved based on the knowledge it obtained and the knowledge of the slave swarms. Furthermore, a PSO approach [25], relying on the functional construction factor and the functional inertia weight, was used to enhance the effect of text feature selection. A hybrid scheme, involving the PSO algorithm [26] and an adaptive learning strategy based on a candidate generation strategy, was designed and applied to establish an effective exploration mechanism and competitive learning based prediction strategy. A genetic learning PSO algorithm (GL-PSO) [27] used a ring topology, to improve the exploration of the search space, and a global learning component with linearly adjustable control parameters to ensure the algorithm adaptability. A hybrid version of PSO and Variable Neighborhood Search was suggested [28] to solve different instances of the constraint shortest path problem. Indeed, two techniques (a novel expanding neighborhood topology and different equation for the particles velocities) were introduced to perform the exploration search.

A dynamic neighbourhood-based switching PSO was developed, using a new velocity updating scheme relying on the adjustment of pbest and gbest in terms of a distance-based dynamic neighbourhood, in order to use all swarm evolution information [29]. To resolve the premature convergence of PSO algorithm, Tolabi et al. integrated different strategies including the division of the search space to find the promote zone, and the use of a probabilistic function to manipulate the inadequate particles [30]. The enhancement of PSO by employing other metaheuristic methods or EAs allows avoiding premature convergence. Likewise, several PSO algorithms that use island were presented to improve optimization process. Based on the previous related studies, it is obvious that the most commonly used strategy is the master-slave model because its swarm procedure yields the best solution.

Table 1
Island model parameters

Parameters	Notation
Migration Topology	T
Islands number	n
Island size	I_s
Migration interval	μ
Selection strategy	S_m
Replacement strategy	R_m
Migration rate	χ

However, this model requires a high computational time to improve global optimization result. More precisely, this process includes the evaluation of different individuals and communication strategies among several slave nodes and the one master node. In addition, a PSO algorithm based on an island model strategy can avoid premature convergence and its performance may be further improved.

In the present work, we propose different kind of island model strategies to examine different communication topologies for the sub-swarms and the migration policy. Besides, the number of island is also modified to verify the effect of the proposed strategies on the PSO performance.

3. Proposed approaches

In the island models, the PSO algorithm is implemented in a parallel way and the migration process among sub-swarms maintains diversity and avoids local minima. We implement several hybridization schemes, including PSO and island models, in different situation such as static and dynamic cases.

3.1. Island model particle swarm optimization

IMPSO algorithm starts the search process by dividing several solutions into sub-swarms $I_1, I_2, I_3, \dots, n$. These sub-swarms are arranged in a T topology migration and the distributed nodes are connected by arcs. Each of them is composed of a set of particles having a position in the search space denoted by $x_i = x_{i1}, x_{i2}, \dots, x_{id}, \dots, x_{iD}$ and it remembers the best position that it met during the process of improving the quality of its position. Each particle position defines a solution to the treated problem. The steps of the introduced approach are described below:

– Initialization phase

The initialization of various parameters presents by: the island number representing the number of sub-swarms (n islands), the size of each island (I_s) where ($n * I_s$) is the size of the IMPSO algorithm. In each island or "sub-swarm", random positions represented by ($x_i = x_{i1}, x_{i2}, \dots, x_{id}, \dots, x_{iD}$) are initialized with their velocities denoted by ($v_i = v_{i1}, v_{i2}, \dots, v_{id}, \dots, v_{iD}$) in the search space of dimension D . Then, the best position encountered by each particle (denoted by $pbest_i = pbest_{i1}, pbest_{i2}, \dots, pbest_{id}, \dots, pbest_{iD}$) by its initial position x_i .

– Update gbest and pbest

Then, applying the objective function, the algorithm calculates the quality (fitness) of each particle to be able to retrieve the best position encountered by each island in dimension D (denoted $gbest_{1n} = gbest_{1n}, gbest_{2n}, \dots, gbest_{dn}, \dots, gbest_{Dn}$). At each iteration, a new position is computed for each particle of the sub-swarm according to its current position, its best position and the global best position encountered by the sub-swarm then the particles move according to the equations (1) and (2): After creating new solutions, the algorithm proceeds to evaluate (i.e. calculate the fitness f for each new particle) and updates values of the $pbest_i$ and also $gbest_n$, as shown in formulas (1) and (2), respectively.

If ($f(x_i)$ is better than $f(pbest_i)$) then $pbest_i = x_i$.

If ($f(pbest_i)$ is better than $f(gbest)$) then
 $gbest = pbest_i$.

– Migration Process

In this sub-section, we study different types of migration topologies used by the island models and the parallel optimization. As instance of the simplest topologies, we can state the chain topology where the islands are arranged in a sequence and only the neighboring islands can communicate together. The communication of the chain topology can be unidirectional or bidirectional. In this study, we examine the first type because it has a specific property which consists in the fact that the first island in the chain does not receive foreign particles. Indeed, when the first island is connected to the last one, the topology becomes unidirectional ring topology. In this case, the communication can be unidirectional or bidirectional. The bidirectional ring topology was used in our experiments together with torus topology. The

latter is generally employed in parallel computing systems having distributed memory.

As far as the migration interval and migration rate, each island evolves independently according to its own dynamics. After performing μ iterations, representing the migration interval, a set of solutions determined by the migration rate χ is selected for migration using an elitist migration strategy named 'best-worst'. We propose another adaptive strategy based on the 'best-worst' and 'best-random' strategies according to a defined probability. This mechanism is used to explore the search space based on the simulated annealing algorithm [31].

Algorithm 1 Island Model Particle Swarm Optimization

```

Initialize island model parameters
Select  $T$  topology
for each island from 1 to  $n$  do
    Initialize the velocity and random position of
    the particles in each dimension of the search
    space
end for
while the stopping criteria is not satisfied do
    for each island from 1 to  $n$  do
        Evaluate  $f(x_i)$  of each particle
        Calculate the new velocity using equation
        (2)
        Find the new position using the equation (1)
        Calculate  $f(x_i)$  of each particle
         $pbest_i = x_i$ 
        Calculate gbest
        After  $\mu$  iterations number
        Choose immigrant particles according to  $\chi$ :
        migration rate
        Apply migration policy
    end for
end while
Collect the best solutions for each island in the
topology
Choose the best solution

```

Once the islands receive all emigrant particles, a replacement strategy is used to generate a new sub-swarm in each island. After receiving the emigrants, all solutions are classified. In fact, the best ones are integrated in the island by removing the same number of solutions (the removed particles are either the worst particles for the best-worst strategy or chosen randomly for the best-random strategy). The adaptive strategy used, in our case, performs

a comparison between the global best solution of the transmitter island and that of the received island. If it is better than the best-worst strategy was applied, if not the best-random strategy was applied with a probability : $pr = e^{\frac{(-\Delta(f(gbest_{I_v}) - f(gbest)))}{Tem}}$ (see algorithm 2).

Algorithm 2 Adaptive strategy Procedure (received island)

```

Initialize  $Tem_0 = Tem$ 
for each  $I_v \in V$  (transmitter islands according to
 $T$  topology) do
  if  $gbest_{I_v}$  dominates  $gbest$  then
    Apply 'best-worst' strategy
  else
    Apply 'best-random' strategy with proba-
    bility  $pr = e^{\frac{(-\Delta(f(gbest_{I_v}) - f(gbest)))}{Tem}}$ 
  end if
end for
 $Tem = \alpha Tem$ 

```

with:

- Tem_0 : the initial temperature,
- Tem : the current temperature,
- $pr = e^{\frac{(-\Delta(f(gbest_{I_v}) - f(gbest)))}{Tem}}$,
- α : it is randomly chosen between 0 and 1

3.2. Dynamic island particle swarm optimization

In this section, we integrate the dynamic island model paradigm based on a learning strategy, proposed in [7], into the PSO algorithm to control dynamically the migration mechanisms, such as the migration rate and the immigrant selection policy, and to maintain the particles diversity in the islands. The dynamic island model which is applied to PSO algorithm can be defined by the following elements:

A migration topology T can be represented by an undirected graph (I, V) where $I = \{I_1, \dots, n\}$.

A migration policy can be represented as a squared matrix M of size n , such that $M(i, j) \in [0, 1]$ represents the possibility for an individual to migrate from island i to island j . i.e., the initialization of M is noted by $\forall(i, j) \in I^2 \setminus V, M(i, j) = 0$. The dynamic migration policy progresses during the search. In fact, each island is equipped with two main processes that define how its sub-swarm evolves and how the migration policy is controlled. These parameters

are modified during the execution according to the incoming feedback received from other islands. This feedback allows the island to know how its particles were improved on other islands. The role of learning component consists in sending more individuals to the island that had been able to improve them significantly and less individuals to islands that are currently less efficient for these particles. The selection component uses the migration matrix M to choose the particles that will be sent to other islands. However, the function of the analysis component is used to evaluate the improvements of the particles after applying the PSO. This component sends this feedback information to the island from which these individuals originated. Figure 1 represents the dynamic island mechanism of the migration policy. The basic processes and data structures are defined below.

A data matrix Dm is a square matrix of size n . $Dm(i, j)$ evaluates the enhancements obtained by the particles of island i when they are processed on island j . An algorithm PSO is used to compute a new swarm I_{t+1} from a given population I_t by applying an evolution process.

Learn (Dm, Dm_{in}) is a process that modifies Dm according to the information received in data input Dm_{in} .

Update (M, Dm) is a process that changes the migration matrix M according to the data matrix Dm . In fact the migration matrix is updated by applying equation:

$$M(i, j) = (1 - \beta)(\alpha M(i, j) + (1 - \alpha)R(j)) + \beta Y(j) \quad (3)$$

where Y is a stochastic vector such that $\|Y\| = 1$. The parameter α represents the importance of the knowledge accumulated during the last migrations and β designates the amount of noise necessary to explore alternative search space areas by means of particles. The parameter R is the intermediate reward vector employed as an intensification strategy according to the update process using the data matrix Dm where only the line corresponding to island i is modified and the particles of the island i having obtained the best improvement are rewarded:

$$R(j) = \begin{cases} \frac{1}{|B|} & \text{if } j \in B \\ 0 & \text{else} \end{cases} \quad (4)$$

with:

$$B = \text{argmax} Dm(i, j) \quad (5)$$

Select (i, M, j) is a process that chooses the sets of particles from the current sub-swarm of the island i

to be sent to another island j according to migration matrix M .

Analyze (i, I, j) is a process that evaluates the improvement obtained in an island j by particles from island i during the last iteration of the algorithm Dm_{out} .

Algorithm 3 Dynamic Island Model Particle Swarm Optimization

```

Initialize the islands
while the stopping criteria is not satisfied do
  for each island do
    Apply the PSO algorithm
    After  $\mu$  iterations number
    Update the transition matrix  $M$ 
    Apply the migration process
  end for
end while

```

3.3. Constrained island model particle swarm optimization

A variant of IMPSO algorithm was applied in the experiments to solve constrained optimization problems. The particles were distributed into islands and conducted throughout the search space to ensure that they would not be trapped in the local minima. Indeed, PSO algorithm was modeled and executed to produce optimality and the evaluation was performed

through the information exchange between the different leaders based on distributed stochastic algorithm mechanism [32]. In this process, the most assessed leader agents migrate from an island to another taking in consideration the violation constraints of the problem.

– Constraint handling techniques

The introduced algorithm combines two constraint-handling techniques for constraint optimization [33], the violation constraints equation and the stochastic ranking procedure. The violation constraints equation [34] prevents the particles of each island from becoming incoherent, while the stochastic classification strategy allows a balanced search in feasible and infeasible regions. The constraint violation function, denoted C , for each particle is presented by the following equation :

$$C = \sum_{k_1=1}^p |h_{k_1}(x)|^r + \sum_{k_2=1}^m \max(0, g_{k_2}(x))^r \quad (6)$$

where $k_1 = 1, 2, \dots, p$ et $k_2 = 1, 2, \dots, m$ are the equality and inequality constraints, while p and m are respectively the numbers of equality and inequality constraints (r is normally set to 1).

Then, Stochastic Ranking procedure proposed by [35] was adapted to dynamically balance the dominance of the feasible and infeasible particles. Each particle of an island was classified according to both the value of the objective function and the constraints violation equation.

The stochastic classification procedure is presented by Algorithm 4, where I_s is the island size, $rand(0, 1)$ denotes a generator of uniform random numbers, f represents the objective function to be minimized and C corresponds to the constraint violation equation. prb is the parameter responsible for the random classification. It decides using either the objective function or the violation constraints function during the comparison phase. Moreover, two adjacent particles x_i and x_{i+1} , where at least one is unfeasible, the comparison of their objective function values is performed with pr probability while the comparison of their violation constraints values is performed with $1 - prb$ probability. When two particles are feasible, $prb = 1$ and the particle having the best objective function value is considered.

– Migration process

The sub-swarms were connected with a migration topology T where each node corresponds to an island.

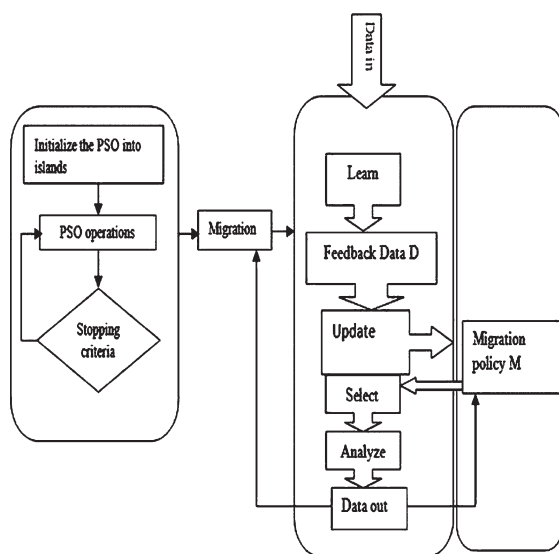


Fig. 1. General mechanism for an Island PSO algorithm used dynamic model.

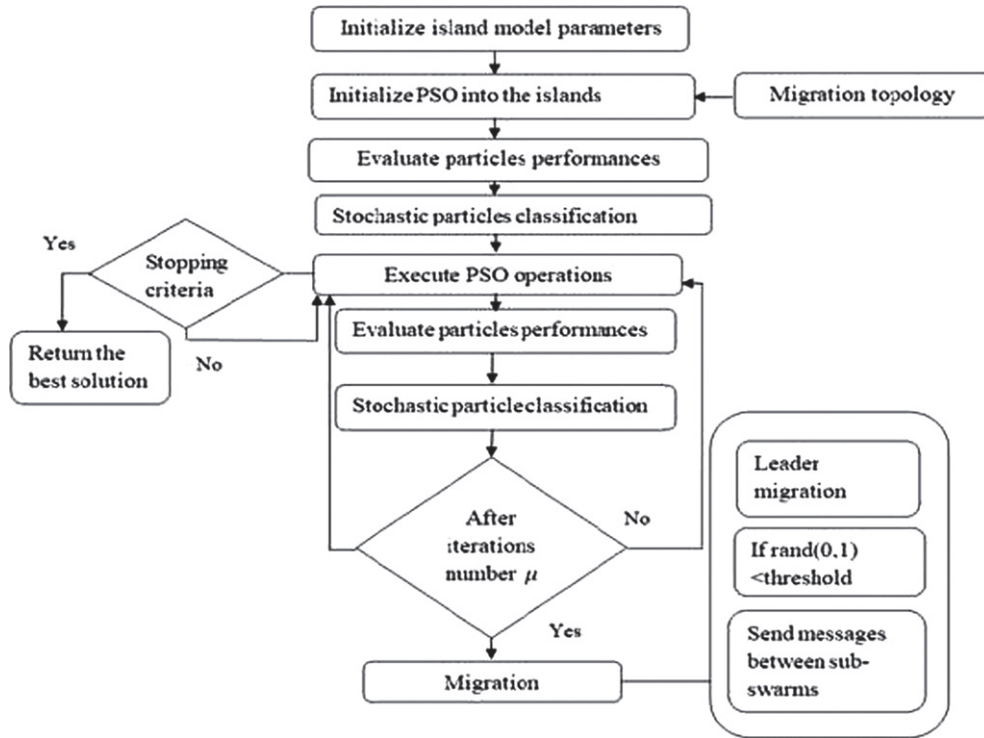


Fig. 2. CIMPSO algorithm flowchart.

Algorithm 4 Stochastic classification algorithm

```

for each island  $I_n$  do
  for  $i = I_s$  to 2 do
    Select a random number  $u \in \text{rand}(0, 1)$ 
    if  $(C(x_{i-1}) = C(x_i) = 0)$  or  $(u < prb)$  then
      if  $(f(x_{i-1}) > f(x_i))$  then
        Permute  $(x_i, x_{i-1})$ 
      else
        if  $(C(x_{i-1}) > C(x_i))$  then
          Permute  $(x_i, x_{i-1})$ 
        end if
      end if
    end if
  end for
end for
  
```

The communication strategy between the islands was applied based on the mechanism of DSA algorithm that improved the optimality produced during the messages exchange between the islands. Furthermore, to reduce the communication messages between islands, only the leaders (the best solutions) were transferred and messages were processed synchronously. After the migration process was triggered

between the islands after μ number of iterations, a value message would be sent to the neighboring island according to the chosen topology. If the random generator number value $\text{rand}[0,1]$ is less than the given threshold, then the message value assignment will be assigned to the island leader after taking into account the number of violated constraints. These messages were communicated on all islands.

Algorithm 5 Adaptive DSA procedure

```

while the migration is triggered do
  Send a valuable message (ID, fitness value, constraint) to the neighboring island
  When receive the message (ID, fitness value, constraint)
  if  $\text{random}(0, 1) < \text{Threshold}$  then
    Assign values
    Update the leader
  end if
end while
  
```

The different steps of the constraint island model particle swarm optimization (CIMPSO) algorithm steps are presented as follows:

- Step 1: Initialize the island model parameters (topology structure, number of islands, migration frequency) and the particles with random positions ($p_{best} = x_i$);
- Step 2: Initialize the particles of each island and evaluate the objective function and the constraints violation of each particle;
- Step 3: Apply the stochastic classification algorithm to the particles;
- Step 4: Find the best solution gbest;
- Step 5: Calculate the new position x_i of each particle using position and speed equations;
- Step 6: Update pbest and gbest for each island;
- Step 7: Apply the migration process according to the migration interval between the islands using the DSA procedure (see algorithm 5) by taking into account these two criteria:
 - If the number of constraints violated by the leader from the receiving island is superior to that of constraints violated by the leader of the sending island
 - If the two communicating islands have the same number of violated constraints;
- Step 8: Evaluate the particles performance of each island and apply the stochastic classification algorithm to the particles.
- Step 9: Return the best solutions until the stop criterion would be reached. Otherwise, return to step 4.

4. Experimental results and discussion

4.1. Experimental setup

In this section, we consider two benchmark functions groups for constrained and unconstrained problems. The first version algorithms (IMPSO and DIMPSO) were tested on 40 benchmark functions, including 25 are taken from test functions CEC'05 (f_1 to f_{25}) [36] for low-dimensional, 15 test functions are taken from the whole test suite for CEC'13 special session on large scale global optimization (f_{26} to f_{40}) for high-dimensional [37]. The test functions, containing a large number of local minima, are non-separable, rotated and multi-modal. The CIMPSO version was tested on 9 test functions (g_{01} to g_{10}) which are selected from the set of constrained optimization problems CEC'06 [38]. These functions can be quadratic, linear or non-linear cubic.

For IMPSO and DIMPSO algorithms, the optimization task was executed 25 times for CEC'05

Table 2
Parameters of the algorithms

Parameters used by S'PSO11	
Swarm size	80 particles
Parameters used by GA	
Population size	100 chromosomes
Selection	Wheel draw
Crossover	$p_c = 0.9$
Mutation	polynomial, $p_m = \frac{1.0}{L}$
Parameters used by IMPSO	
Swarm size	80 particles
Islands number	3, 5, 10
Migration topology	Chain, ring, torus
Parameters used by DIMPSO	
Swarm size	80 particles
Islands number	3, 5, 10
Migration topology	Fully connected
Parameters used by CIMPSO	
Swarm size	80 particles
Migration topology	Chain, ring, torus
Islands number	5, 10

and CEC'13 test functions and the obtained results were illustrated in the form of average error of the best solution $\|f(x^*) - f(x^{best})\|$ noted that x^* is a given optimal solution while the x^{best} is the average best solution obtained in 25 runs. By applying the CIMPSO algorithm, the average results are obtained after 25 runs of the best solution found for each constraint optimization problem.

4.2. Unconstrained test functions

The parameters used in the island models were intensively studied to measure their effects on the convergence rate and diversity of the proposed algorithms. The parameters employed in the IMPSO present by as follows: the number of islands [39] n used with various values $\{n = 3, 5, 10\}$. They are studied in the three cases with double ring topology, chain topology and torus topology. According to research study [6] shows that they are the best topologies applied for the Genetic Algorithm. The other major parameters utilized in island models are migration interval (μ iterations) and migration rate (χ) denoting respectively how much frequently migrations occur and how many particles migrate. The considered interval migration is $\mu = 100$ iterations number and the number of emigrant is 10% chosen according to the best-worst replacement strategy and the adaptive strategy. Then, the parameters used by DIMPSO are the studied island number $\{n = 3, 5, 10\}$ values and the values of α and β are respectively 0.8 and 0.01 [40].

Table 3
Average error rate obtained by IMPSO using chain topology

	Best-worst strategy			Adaptive strategy			p-value
	n=3	n=5	n=10	n=3	n=5	n=10	
f_1	1.23E-008	1.92E-008	0.0	1.24E-009	0.0	0.0	+
f_2	1.34E-008	1.28E-009	9.94E-009	1.73E-009	1.63E-009	1.33 E-009	+
f_3	1.75E-009	1.58E-009	1.63E-009	1.24E-009	1.14E-009	1.12E-009	+
f_4	2.63E-009	1.87E-009	0.0	1.68E-009	1.46E-009	0.0	+
f_5	1.97E-005	1.73E-005	1.01E-009	1.52E-008	1.00 E-009	1.16E-009	+
f_6	1.26E-009	1.34E-009	1.51E-11	1.45E-006	1.56E-006	1.38E-10	+
f_7	4.57E-002	5.26E-002	3.95E-01	2.31E-002	1.14E-002	3.42E-003	+
f_8	2.2E+001	2.06E+001	2.00E+001	2.22E+001	2.11E+001	1.98E+001	-
f_9	1.38E-001	1.27E-001	2.30E-001	2.37E-001	2.31E-001	1.26E-001	+
f_{10}	1.43E+000	1.57E+000	1.27E+000	1.28E+000	1.24E+000	1.43E+000	+
f_{11}	2.82E-001	2.72E-001	2.86E-002	1.79E-002	1.63E-002	2.85E-002	+
f_{12}	9.46E-001	8.54E-001	8.88E-001	8.94E-001	8.85E-001	8.26E-001	+
f_{13}	1.35E+000	1.43E+000	1.24E+000	1.53E+000	1.12E+000	1.23E+000	+
f_{14}	2.28E+000	2.65E+000	3.45E+000	2.42E+000	2.27E+000	2.34E+000	+
f_{15}	5.61E+000	4.75E+000	4.82E+000	5.23E+000	4.84E+000	4.92E+000	-
f_{16}	2.42E+000	2.33E+000	2.41E+000	2.64E+000	2.32E+000	2.39E+000	+
f_{17}	3.86E+000	3.28E+000	3.79E+000	3.98E+000	3.76E+000	3.83E+000	+
f_{18}	1.32E-006	1.39E-007	4.91E-008	4.56E-008	4.48E-008	4.34E-008	+
f_{19}	1.41E-006	1.32E-006	1.67E-005	1.24E-005	1.92E-005	1.57E-005	+
f_{20}	6.93E-007	5.38E-007	5.44E-008	5.89E-007	5.68E-008	5.49E-008	+
f_{21}	1.44E-006	1.23E-006	1.99E-007	1.42E-006	1.25E-006	1.38E-007	+
f_{22}	6.24E+001	7.53E+001	7.86E+001	6.35E+001	7.58E+001	7.53E+001	+
f_{23}	2.25E+000	2.16E+000	2.28E+000	2.43E+000	2.36E+000	2.78E+000	+
f_{24}	8.76E-001	8.72E-001	9.55E-001	8.58E-001	8.76E-001	9.44E-001	+
f_{25}	1.84E+001	1.79E+001	1.92E+001	1.93E+001	1.86E+001	1.87E+001	-

The parameters used in PSO are the same used in SPSO [41]: a swarm size equal to 80 particles; random initialization of particle positions and velocities; random topology with $K = 3$ informants; acceleration coefficients c_1 and c_2 equal to $0.5 + \ln(2)$ and a constant inertia weight equal to $w = \frac{1}{(2 * \ln(2))}$. (see Table 2) To validate the efficiency and robustness of IMPSO and DIMPSO algorithms, different assessment algorithms were tested: standard PSO, GA [42] and coral reefs optimization algorithm [43] utilized to solve low-dimensional optimization. Besides, Differential Evolution with Cooperative Co-evolution-Differential Grouping (DECC-dg2) [44] and Social Learning-Particle Swarm Optimization (SL-PSO) [45] were applied to solve large-scale optimization problems. The maximum number of iterations is fixed to 250 for the different algorithms. They are studied in the three cases with double ring topology, chain topology and torus topology.

4.2.1. Performance study of the test functions CEC'05

The average error results obtained by the IMPSO and DIMPSO algorithms for all 25 problems are reported in Tables 3-6. It is clear that, for unimodal functions (f_1 , f_2 , f_3 , f_4 and f_5) based on 10-D, the IMPSO algorithm provided better results than the

standard PSO, for 4 functions out of 5 functions. It is also obvious that DIMPSO gave better results than the IMPSO algorithm, considering the different used topologies (chain, ring and torus) and the two models of the migration policy.

For multi-modal functions (f_6 to f_{12}) containing several local minima, the IMPSO algorithm was more guided towards the optima than the DIMPSO for all functions. It can be noticed, from the tables above, that, when the bidirectional ring and torus topologies are used, the average error values of each problem decreased. The only exception is f_8 problem characterized with the optima on the boundary. As the two proposed approaches were compelled in the boundary, it was hard to find the solution on the boundary. For the third set functions f_{13} and f_{14} , the two algorithms showed competitive results.

For the hybrid functions ($f_{15} - f_{25}$), the IMPSO provided almost the better average errors, compared to DIMPSO algorithm. Obviously, it is better to use higher value of n islands in the two approaches. In other words, when the value of n is small, the particles in the IMPSO and DIMPSO are not sub-divided into several islands and, thus, few search space regions will be simultaneously explored. However, a large number of n lead to a better diffusion in the swarm. The obtained findings show that the increase in

Table 4
Average error rate obtained by IMPSO using ring topology

	Best-worst strategy			Adaptive strategy			p-value
	n=3	n=5	n=10	n=3	n=5	n=10	
f_1	1.23E-013	0.0	0.0	1.28E-009	1.25E-009	0.0	+
f_2	1.31E-009	1.25E-009	1.11E-009	1.22E-009	1.14E-009	0.0	+
f_3	1.44E-009	1.31E-009	1.27E-009	1.31E-009	1.23E-009	1.47E-009	+
f_4	1.15E-009	0.0	0.0	1.38E-009	1.27E-009	1.44E-009	+
f_5	1.64E-007	1.27E-009	1.35E-009	1.29E-008	1.12E-008	1.26E-009	+
f_6	1.23E-009	1.42E-10	1.56E-11	1.38E-009	1.21E-10	1.33E-10	+
f_7	3.67E-002	3.46E-002	2.85E-001	2.34E-001	2.46E-002	2.58E-002	+
f_8	2.16E+001	1.97E+001	1.89E+001	2.23E+001	2.00E+001	2.17E+001	-
f_9	1.49E-001	1.38E-001	2.57E-001	1.37E-001	1.45E-001	1.76E-001	+
f_{10}	1.38E+000	1.53E+000	1.82E+000	1.56E+000	1.43E+000	1.68E+000	+
f_{11}	2.95E-002	2.41E-002	2.54E-002	2.87E-002	2.64E-002	2.85E-002	+
f_{12}	7.84E-001	7.75E-001	8.90E-001	8.39E-001	8.35E-001	8.23E-001	+
f_{13}	1.47E+000	1.23E+000	1.17E+000	1.32E+000	1.18E+000	1.24E+000	+
f_{14}	2.83E+000	2.72E+000	2.43E+000	2.52E+000	2.49E+000	2.37E+000	+
f_{15}	4.96E+000	4.84E+000	4.53E+000	4.96E+000	4.67E+000	4.89E+000	-
f_{16}	2.45E+000	2.24E+000	2.37E+000	2.64E+000	2.53E+000	2.71E+000	+
f_{17}	3.87E+000	3.69E+000	3.75E+000	4.37E+000	4.25E+000	3.63E+000	+
f_{18}	4.98E-008	4.69E-008	4.81E-008	3.53E-007	4.46E-008	5.27E-008	+
f_{19}	1.85E-006	1.72E-006	1.54E-006	1.67E-006	1.42E-006	1.25E-006	+
f_{20}	5.76E-007	5.34E-008	5.57E-008	4.48E-007	4.27E-007	4.34E-007	+
f_{21}	2.25E-007	2.18E-007	1.95E-007	2.57 E-007	2.62E-007	2.78 E-007	+
f_{22}	6.88E+001	6.75E+001	7.01E+001	6.74E+001	6.61E+001	6.79E+001	+
f_{23}	2.26E+000	2.24E+000	2.00E+000	2.35E+000	2.26E+000	2.42E+000	+
f_{24}	9.63E-001	9.59E-001	9.45E-001	9.45E-001	9.87E-001	9.93E-001	+
f_{25}	2.24E+001	1.82E+001	1.95E+001	2.23E+001	2.15E+001	2.28E+001	-

Table 5
Average error rate obtained by IMPSO using torus topology

	Best-worst strategy		Adaptive Strategy		p-value
	n=5	n=10	n=5	n=10	
f_1	1.25E-009	0.0	1.37E-10	0.0	+
f_2	1.34E-009	0.0	1.23E-009	1.85E-10	+
f_3	1.31E-009	1.63E-009	1.95E-009	2.32E-008	+
f_4	1.17 E-009	0.0	1.36E-009	1.15E-009	+
f_5	1.83E-008	1.28E-009	2.18E-009	2.27E-009	+
f_6	1.27E-10	1.51E-11	1.34E-009	1.23E-10	+
f_7	3.56E-003	3.83E-003	3.26E+003	3.45E-003	+
f_8	2.24E+001	2.00E+001	2.28E+001	2.35E+001	-
f_9	2.1E-001	2.26E-001	2.43E-001	2.52E-001	+
f_{10}	1.28E+000	1.36E+000	1.34E+000	1.44E+000	+
f_{11}	2.75E-002	2.8E-002	2.53E-002	2.69E-002	+
f_{12}	8.69E-001	8.98E-001	7.46E-001	7.25E-001	+
f_{13}	1.34E+000	1.29E+000	1.37E+000	1.28E+000	+
f_{14}	3.62E+000	3.23E+000	3.41E+000	3.58E+000	+
f_{15}	5.23E+000	5.37E+000	5.14E+000	5.39E+000	-
f_{16}	2.64E+000	2.49E+000	2.38E+000	2.57E+000	+
f_{17}	3.42E+000	3.55E+000	3.73E+000	3.64E+000	+
f_{18}	4.88E-008	4.97E-008	3.85E-008	4.82E-008	+
f_{19}	1.56E-004	1.4E-004	2.36E-004	1.38E-004	+
f_{20}	4.73E-008	4.89E-008	4.82E-008	5.23E-008	+
f_{21}	2.41E-006	2.23E-007	2.43E-007	2.81E-007	+
f_{22}	5.24E+001	6.81E+001	6.24E+001	6.65E+001	+
f_{23}	2.52E+000	2.46E+000	2.47E+000	2.64E+000	+
f_{24}	9.38E-001	9.88E-001	9.98E-001	9.67E-001	+
f_{25}	2.26E+001	1.99E+001	2.32E+001	2.26E+001	-

Table 6
Average error rate obtained by DIMPSO and SPSO'11

	DIMPSO			SPSO'11	p-value
	n=3	n=5	n=10		
f_1	1.12E-009	0.0	2.13E-008	1.00E-009	+
f_2	1.23E-009	0.0	1.21E-009	1.00E-009	+
f_3	1.09E-009	0.0	1.48E-009	1.00E-009	+
f_4	8.76E-009	0.0	1.85E-003	1.52E-004	+
f_5	1.84E-006	0.0	2.23E-006	1.6E-005	+
f_6	1.31E-009	1.35E-007	4.45E-007	4.2E-007	+
f_7	4.36E-002	3.36E-001	4.26E-001	5.1E-002	+
f_8	2.19E+001	1.86E+001	21.00E+001	1.94E+001	-
f_9	2.24E-001	2.25E-001	1.13E-009	1.17E-009	+
f_{10}	1.87E+000	1.7E+000	2.78E+000	3.2E+000	+
f_{11}	2.63E-001	1.35E-0001	3.48E+000	2.31E+001	+
f_{12}	9.74E-001	8.9E-001	3.12E+000	1.89+000	+
f_{13}	1.68E+000	1.14E+000	2.63E+000	1.89E-001	+
f_{14}	3.47E+000	4.3E+000	3.35E+000	3.65E+000	+
f_{15}	5.52E+000	4.79E+000	4.85E+000	4.29E+000	-
f_{16}	3.44E+000	2.33E+000	8.32E+000	7.12E+000	+
f_{17}	4.26E+000	4.13E+000	3.12E+001	2.21E+001	+
f_{18}	3.51E-007	4.01E-007	2.89E+000	2.67E+000	+
f_{19}	1.87E-002	1.78E-005	6.28E+000	4.21E+002	+
f_{20}	5.23E-008	7.04E-007	4.52E+001	4.74E+002	+
f_{21}	4.64E-007	2.86E-007	4.75E+002	2.37E+002	-
f_{22}	8.49E+001	4.86E+002	583E+002	5.45E+002	+
f_{23}	2.33E+000	1.8E+000	5.81E+002	4.21E+001	-
f_{24}	9.69E-001	9.38E-001	2.16E+001E	1.09E+002	+
f_{25}	2.16E+001	2.85E+001	2.51E+002	2.56E+002	-

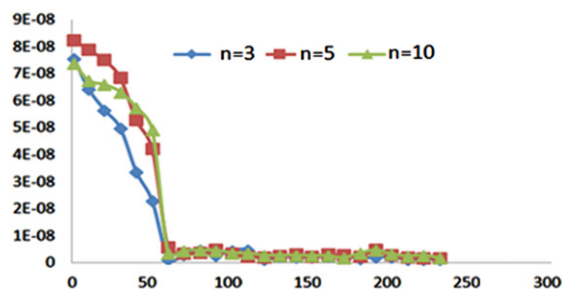


Fig. 3. Graph convergence of the f_6 Shifted Rosenbrock's Function obtained by IMPSO (ring topology, best-worst strategy).

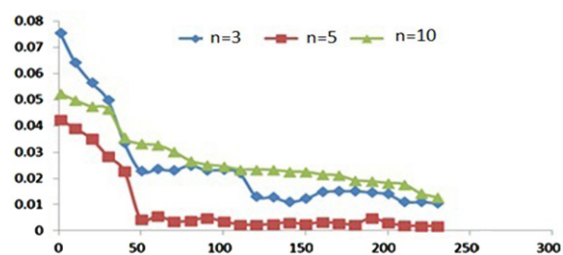


Fig. 4. Graph convergence of the f_7 Shifted Rotated Griewank's Function without Bounds obtained by IMPSO using ring topology (ring topology, best-worst strategy).

the value of n did not only improve the obtained results, but also rose the computational complexity of the algorithm. For this reason, the employment of different islands number, whether for IMPSO or DIMPSO, enhanced the convergence and diversity of the PSO. As the topology parameter played an important role in ameliorating the PSO performance, the ring topology provided better results as it uses the best-worst replacement strategy in most of the test functions, compared to other topologies (chain and torus). Besides, the adaptive strategy yielded good results, especially for the chain topology. It is to note that the migration process occurs using unidirectional communication between islands.

The obtained average errors of the convergence rate show that IMPSO is a winner as it applies three topologies (chain, ring, torus) especially for multi-modal test functions. These different migration topologies provide wide amplitude of displacement of the immigrant solutions between the islands that favor the exploration of the research space. Figures 3 and 4 reveal the convergence graph for f_6 and f_7 obtained by IMPSO. Figures 5 and 6 illustrate the trajectory of a particle for shifted rotated ackley's function obtained by IMPSO and DIMPSO algorithms. It appears that the particle is moving from one extremity to another, hence the search is almost seeking on bounds. That is why the IMPSO and DIMPSO

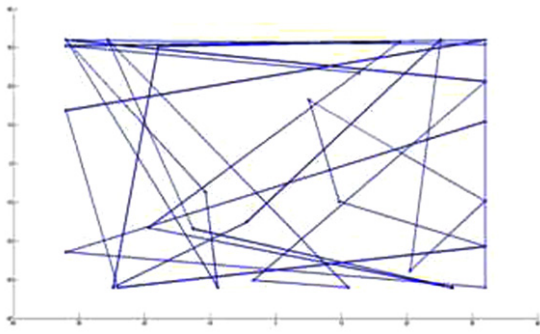


Fig. 5. Trajectory of a particle for Shifted Rotated Ackley's Function obtained by IMPSO ($n=5$, ring topology, best-worst strategy).

failed in solving this problem in all 25 runs. Moreover, Figures 7 and 8 expose the position evolution of the best position of the swarm, in terms of the number of iterations for the test functions f_9 , in the three cases according to the number of islands. They show that the best particle converge to a position in the search space using the two approaches. A statistical testing phase was applied in the performed experiments in order to make the results more reliable using a multi-comparative function ANOVA. A confidence level of 95 % (i.e. significance level of 5% or p-value below 0.05) was considered in the statistical tests. Successful tests are marked with '+' symbols in the last column of the tables representing the average error, while unsuccessful tests are marked with '-' symbols showing no statistical confidence (p-value > 0.05). Therefore, we can observe that the DIMPSO algorithm provided good results for the unimodal test functions. Moreover, IMPSO algorithm yielded good findings when applied in almost all test functions.

4.2.2. Comparison with other evolutionary algorithms

In this section, the performance of the proposed approaches is studied using the set benchmark functions CEC'05 (f_1 to f_{25}) in dimension 30. The different applied algorithms are presented as follows:

- S'PSO11
- IMPSO: Indeed, the migration policy is the 'best-worst' ring migration topology and the number of islands is equal to 5.
- DIMPSO: is a PSO algorithm relying on dynamic island model strategies where the number of islands is equal to 5.
- GA: a genetic algorithm which implements a uniform crossover and a wheel draw selection.

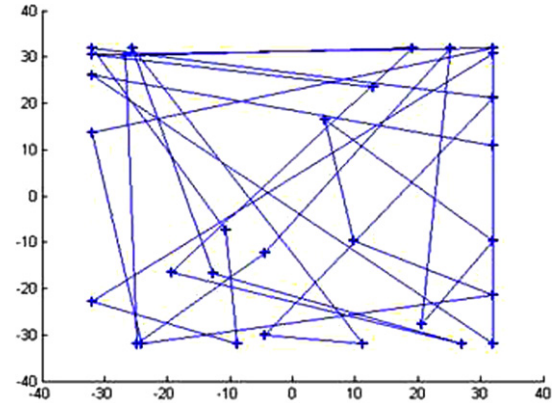


Fig. 6. Trajectory of a particle for Shifted Rotated Ackley's Function obtained by DIMPSO ($n=5$).

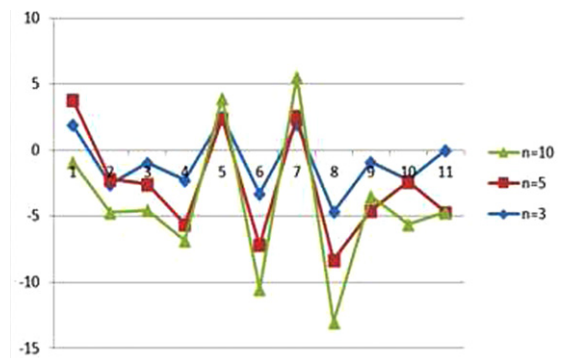


Fig. 7. Vector position of the best particle of the swarm for the Rastrigin function (f_9) obtained by IMPSO ($n=5$, ring topology, best-worst strategy).

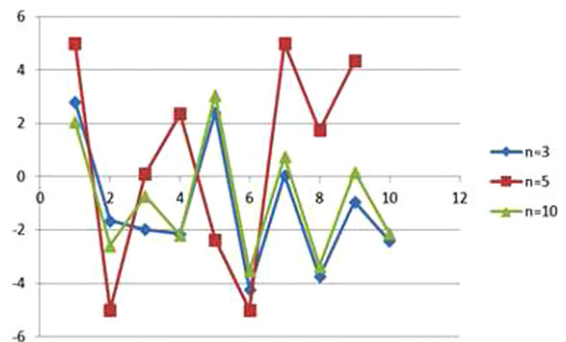


Fig. 8. Vector position of the best particle of the swarm for the Rastrigin function (f_9) obtained by DIMPSO ($n=5$).

- CRO: The parameters of the Coral Reefs Optimization algorithm are presented as follows: the reef size is equal to 100, multi-point crossover,

Table 7
Average error rate results of the 25 test functions in 30-D

	IMPSO	DIMPSO	S'PSO11	GA	CRO
f_1	0.0	0.0	3.26E-001	1.31E-005	2.43E-008
f_2	1.35E-009	0.0	2.34E-008	1.76E-008	2.78E-008
f_3	1.14E-009	1.00E-009	1.47E-001	1.21E+001	3.32E-007
f_4	0.0	1.23E-007	1.13E-005	1.37E-008	2.21 E-003
f_5	1.47E-009	1.64E-009	1.21E-005	3.47E-005	1.85E-003
f_6	1.23E-10	1.35E-006	1.55E-006	1.79E+000	2.37E-005
f_7	1.42E-001	2.76E-001	3.64E-001	1.92E-001	3.32E-001
f_8	2.33E+001	1.95E+001	2.01E+001	20.85E+001	2.26E+001
f_9	1.71E-008	3.54E-008	9.27E-007	2.96E-001	1.24E+000
f_{10}	1.94E+000	2.19E+000	2.78E+000	5.30E+000	2.86E+000
f_{11}	2.54E-001	1.78E-001	2.54E+001	1.97E+000	1.73E+000
f_{12}	2.00E+000	1.32+000	1.67+000	2.53E+001	1.87E+000
f_{13}	1.26E+000	1.66E+000	1.95E-001	4.28E-001	5.66E-001
f_{14}	2.88E+000	3.28E+000	3.37E+000	3.25E+000	3.43E+000
f_{15}	5.74E+000	4.83E+000	5.24E+000	5.65E+000	5.27E+000
f_{16}	3.86E+000	3.93E+000	4.12E+000	4.37E+000	2.36E+000
f_{17}	4.75E+000	4.42E+000	2.63E+000	2.52E+000	2.47E+000
f_{18}	3.92E-001	4.26E-001	1.87E+000	1.74E+000	2.84E-001
f_{19}	6.41E-002	6.58E-002	5.34E+000	2.45E+001	1.41E-001
f_{20}	2.35E-002	2.26E-002	3.73E+001	3.67E+001	2.62E-001
f_{21}	9.65E-001	9.84E-001	73.58E+001	2.94E+001	9.57E-001
f_{22}	3.47E+001	5.67E+001	5.64E+001	5.79E+001	3.88E+001
f_{23}	2.86E+000	2.71E+000	4.49E+001	4.38E+001	3.26E+000
f_{24}	3.24E+001	3.53E+001	2.38E+001	2.74E+001	2.74E+001
f_{25}	2.25E+001	2.36E+001	2.46E+001	2.65E+001	2.52E+001

Gaussian mutation and the maximum number of iterations 250.

Table 7 presents the average error resulting from 25 test functions obtained by comparative algorithms. The best values are written in bold after 25 executions of each algorithm. Obviously, the two algorithms (IMPSO and DIMPSO) provide the best results for 20 functions out of 25. It is also clear that Sphere, Schwefel and Rosenbrock functions provided the best results in terms of convergence rate. However, the GA and CRO gave mixed results for almost test functions. These results confirm the relevance of the proposed methods in maintaining the diversity.

4.2.3. Performance effect on optimization of high-dimensional problems

When applied to optimize low-dimensional (10-D and 30-D) problems, IMPSO and DIMPSO have shown robust performance on 25 test functions, in comparison with S'PSO'11, GA and CRO. In this section, we discuss their the performance of these algorithms to solve large-scale optimization problems, where the search dimensionality is normally larger than 100. The dimension was set to D-1000. Correspondingly, the island number and the particles number were set respectively to 5 and 100 and the

maximum iterations number was considered as the stopping criteria.

To compare the proposed approaches performance, two popular algorithms were adopted: DECC-dg2 and SL-PSO. The two algorithms were applied to solve large-scale optimization problems in the corresponding papers. DECC-dg2 is an improved DECC algorithm with differential evolution strategy. On the other hand, SL-PSO was efficiently used to solve large-scale optimization problems.

The test results presented in Table 8 show that, on three fully-separable functions (f_{26} to f_{28}), IMPSO yielded the best performance on f_{26} and f_{27} , while DIMPSO was the most performing when applied on f_{28} . Compared with IMPSO, DIMPSO provided good results on two functions. However, its performance was low on one function. From the obtained results, we may conclude that IMPSO performed better than DIMPSO. On eight partially and additively separable functions (f_{29} to f_{36}), IMPSO and DIMPSO were the most efficient on almost benchmark functions. On overlapping functions (f_{37} to f_{39}), IMPSO and DIMPSO achieved the best performance on two functions. On non-separable function (f_{40}), DIMPSO had better performance than IMPSO. For the 15 large-scale optimization functions, IMPSO and DIMPSO win on nine benchmark functions,

Table 8
Average errors on CEC'13 functions

	IMPSO	DIMPSO	SL-PSO	DECC-dg2
f_{26}	4.28E-014	3.76E-013	3.70E-014	8.65E+010
f_{27}	3.89E+003	4.51E+003	6.70E+0037	1.41E+004
f_{28}	2.28E+001	2.25E+001	2.16E+001	2.06E+001
f_{29}	1.15E+010	1.23E+010	1.20E+010	2.51E+008
f_{30}	6.84E+005	6.57E+005	7.58E+005	2.74E+006
f_{31}	1.24E+006	1.18E+006	1.06E+006	1.06E+006
f_{32}	3.42E+006	3.73E+006	1.73E+007	8.93E+007
f_{33}	9.78E+013	1.15E+014	2.89E+014	1.01E+014
f_{34}	5.39E+007	4.95E+007	4.44E+007	3.08E+008
f_{35}	9.40E+007	9.38E+007	9.43E+007	9.44E+007
f_{36}	7.73E+008	7.72E+008	9.98E+009	9.93E+009
f_{37}	1.61E+003	4.22E+003	1.13E+003	5.81E+007
f_{38}	3.92E+008	3.85E+008	2.05E+009	6.03E+008
f_{39}	9.46E+008	9.54E+008	1.60E+010	1.11E+009
f_{40}	7.21E+007	7.16E+007	6.68E+007	7.11E+006

compared to SL-PSO and DECC-dg2. Therefore, the proposed approaches exhibited powerful ability in solving large-scale optimization problems. It is also clear that the different island model strategies are feasible and effective in improving the PSO ability to balance diversity and convergence.

4.3. Constrained test functions

The same parameters of island model employed by IMPSO approach were used and the number of islands was fixed to $n = \{5, 10\}$. The used migration policy presented by the communication strategy of the DSA algorithm, the threshold value was set to 0.2 and the comparison probability prb was set to 0.45. Moreover, Covariance Matrix Adaptation-Evolution Strategy (CMA-ES) algorithm [46], the population size and the evaluation number of the objective function were fixed at 100 and 20,000 evaluations, respectively.

The aim of the conducted experiment is to evaluate the performance of the proposed CIMPSO algorithm

using nine constrained testing functions were selected from CEC'06 benchmark ($g01, g02, g03, g04, g05, g07, g08, g09$ and $g10$).

In terms of the average best value provided by the objective function, the results given by CIMPSO are better than those obtained by applying the CMA-ES algorithm. As shown in table 9, the CIMPSO based on double ring communication topology gave better results, for 5 test functions ($g02, g03, g04, g05, g10$), than the two other topologies. Additionally, the increase in the number of islands allowed CIMPSO algorithm to approach to the global optimum. Therefore, the integration of the island model techniques as well as the communication strategy of DSA algorithm improved the search process on most benchmark functions, except for two benchmark functions ($g01, g08$) where the CMA-ES algorithm reached almost the global optimum. As shown, in Table 8, the obtained results are statically different with 95% confidence, as demonstrated by ANOVA statistical test.

5. Conclusion

This paper presented different distributed versions of island-PSO algorithm used to solve constrained and unconstrained optimization problems. The IM techniques were embedded into the PSO algorithm in order to combine the convergence rate of PSO with the enhanced exploration performed by the IM parameters. The first version (IMPSO) integrates the different island model parameters: migration topology, selection strategy replacement strategy, migration frequency and migration rate. The IM consists in launching several sub-populations by applying PSO algorithm and the islands which evolve independently but interact at certain number of iterations.

Table 9
Average best value of the objective function

	Chain topology		Ring topology		Torus topology		CMA-ES	p-val
	n=5	n=10	n=5	n=10	n=5	n=10		
g01	-14.812	-14.875	-14.832	-14.795	-14.751	-14.743	-14.881	+
g02	-0.796	-0.801	-0.8000	-0.788	-0.778	-0.775	-0.786	+
g03	-0.916	-0.925	-0.989	-0.994	-0.986	-0.991	-0.981	+
g04	-30538.49	-30567.58	-30592.12	-30636.74	-30639.25	-30641.32	-30623.71	+
g05	5124.84	5124.36	5125.61	5125.95	5124.67	5124.3	5125.46	+
g07	25.94	25.73	25.24	25.61	26.53	26.82	25.82	+
g08	-0.0954	-0.0955	-0.0957	-0.0958	-0.0955	-0.0956	-0.0958	+
g09	682.79	682.55	681.22	681.96	683.18	683.45	681.69	+
g10	7124.33	7110.85	7252.62	7237.41	8224.73	8325.28	7141.52	+

We studied the effect of the migration topology, island number and migration policy using a static tuning parameters on the performance of PSO algorithm. The second version (DIMPSO) consists in combining the PSO with dynamic IM that dynamically monitored its performance and used the migration parameters, such as the immigrant number of particles and the immigrant selection policy, in order to improve the performance of the algorithm. The studied topology was represented by a graph. Similarly, the effect of the island number on PSO efficiency was examined. The CIMPSO version was applied to solve constrained problems. The DSA communication strategy among islands was integrated to exchange the best solution based on the violation constraints of the optimization problem. The experiment results showed systematically that the proposed algorithms yielded better results according to the average error for almost test functions. Thus, the results provided by IMPSO and DIMPDO applied to CEC'05 for low-dimension problems demonstrated a good improvement in terms of convergence and diversity, and applied to CEC'13 for high-dimension problems show a promising performance search. The CIMPSO showed good performance for constrained testing functions CEC'06. Therefore, the comparison results reveal that the proposed approaches were efficiently applied to solve low and large-scale optimization problems. They also prove that the different island model strategies are efficiently used to improve the PSO ability to balance convergence and diversity. In our future works, we will apply the developed approaches to classify the massive large-scale, spatial-temporal dependent and uncertain data. In this area, image classification has many problems such as feature selection and parameter optimization. We intend also to propose migration policies and examine their impact on the efficiency of PSO island. The island techniques will be investigated to further improve the multi-objective PSO.

References

- [1] J. Kennedy and E.C. Eberhart, Particle swarm optimization, in *International Conference on Neural Networks, IEEE Service Center Piscataway, NJ, USA, 2005*.
- [2] M. Dorigo and L.M. Gambardella, Ant colony system: a cooperative learning approach to the traveling salesman problem, in *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, April 1997, pp. 53–66. doi: 10.1109/4235.585892.
- [3] D. Karaboga and B. Basturk, Artificial bee colony (ABC) optimization algorithm for solving constrained optimization problems, In: Melin, P., Castillo, O., Aguilar, L.T., Kacprzyk, J., Pedrycz, W. (eds.) *IFSA 2007. LNCS (LNAI)*, vol. 4529, p. 789. Springer, Heidelberg, 2007.
- [4] X.S. Yang and S. Deb, Cuckoo Search via Levy Flights, *IEEE World Congress on Nature and Biologically Inspired Computing*, 2009, pp. 210–214.
- [5] S. Mirjalili, Dragonfly algorithm: a new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems, *Neural Comput and Applic* 27 (2016), 1053–1073. <https://doi.org/10.1007/s00521-015-1920-1>
- [6] M. Rucinskia, D. Izzo and F. Biscania, On the Impact of the Migration Topology on the Island Model, *Parallel Computing Journal* (36) (2010), 555–571.
- [7] C. Candan, A. Goeffon, F. Lardeux and F. Saubien, A Dynamic Island Model for Adaptive Operator Selection, *Proc in GECCO*, 2012, pp. 1253–1260.
- [8] M.A. Al-Betar, M.A. Awadallah, A.T. Khader and Z.A. Abdalkareem, Island-based harmony search for optimization problems, *Journal of Expert Systems with Applications* 2015, pp. 2026–2035.
- [9] F. van den Bergh and A.P. Engelbrecht, A convergence proof for the particle swarm optimiser, *Fundamenta Informaticae* 105 (2010), 341–374.
- [10] M.R. Bonyadi, Z. Michalewicz and X. Li, An analysis of the velocity updating rule of the particle swarm optimization algorithm, *Journal of Heuristics* 20 (2014), 417–452.
- [11] M.Z. Bonyadi and Z. Michalewicz, Particle swarm optimization for single objective continuous space problems: a review, *Evolution Computation*: 2017, pp. 1–54.
- [12] Y.J. Gong, J. Zhang, H. Chung, W. Chen, Z.H. Zhan, Y. Li and Y. Shi, An efficient resource allocation scheme using particle swarm optimization, *IEEE Transaction Evolution Computation*: 2012, pp. 801–816.
- [13] R. Chakraborty, R. Sushil and M.L. Garg, An Improved PSO-Based Multilevel Image Segmentation Technique Using Minimum Cross-Entropy Thresholding, *Arabian Journal for Science and Engineering* (44) (2019), 3005–3020.
- [14] M.S. Hasanoglu and M. Dolen, Feasibility enhanced particle swarm optimization for constrained mechanical design problems, *Journal of Mechanical Engineering Science*: 2018, pp. 381–400.
- [15] F.Q. Lu, Huang, W.K. Ching and T.K. Siu, Credit portfolio management using two-level particle swarm optimization, *Information Science*: 2013, pp. 162–175.
- [16] A.A. Esmine, R.A. Coelho and S. Matwin, A review on particle swarm optimization algorithm and its variants to clustering high-dimensional data, *Artificial Intelligence*, 2015, pp. 23–45.
- [17] B. Niu, Y. Zhu, X. He and H. Wu, MCPSO: a multi-swarm cooperative particle swarm optimizer, *Applied Mathematics and Computation* 185(2) (2007), 1050–1062.
- [18] M.S. Kiran and M. Gündüz, A recombination-based hybridization of particle swarm optimization and artificial bee colony algorithm for continuous optimization problems, *Applied Soft Computing Journal* 13 (2013), 2188–2203.
- [19] Y.C. Lu, J.C. Jan, S.L. Hung and G.H. Hung, Enhancing particle swarm optimization algorithm using two new strategies for optimizing design of truss structures, *Engineering Optimization Journal* 45 (2012), 1251–1271.
- [20] J. Shobana and M. Murali, Adaptive Particle Swarm Optimization Algorithm Based Long Short-term Memory

- Networks for Sentiment Analysis, *Journal of Intelligent and Fuzzy Systems* 1 Jan. 2021, pp. 10703–10719.
- [21] Z. Zhang, X. Li, S. Luan and Z. Xu, An efficient particle swarm optimization with homotopy strategy for global numerical optimization, *Journal of Intelligent and Fuzzy Systems* **40**(3) (2021), 4301–4315. DOI: 10.3233/JIFS-200979
- [22] C. Qiu and N. Liu, A Novel Three Layer Particle Swarm Optimization for Feature Selection, *Journal of Intelligent and Fuzzy Systems* **41**(1) (2021), 2469–2483. DOI: 10.3233/JIFS-202647
- [23] M. Al-Andoli, W.P. Cheah and S.C.Tan, Deep Autoencoder-based Community Detection in Complex Networks with Particle Swarm Optimization and Continuation Algorithms, *Journal of Intelligent and Fuzzy Systems* **40**(3) (2021). DOI: 10.3233/JIFS-201342
- [24] K.E. Parsopoulos, Parallel cooperative micro-particle swarm optimization: A master slave model, *Journal of Applied Soft Computing* **12** (2012), 3552–3579.
- [25] L. Yonghe, L. Minghui, Y. Zeyuan and C. Lichao, Improved particle swarm optimization algorithm and its application in text feature selection, *Applied Soft Computing* **35** (2015), 629–636.
- [26] F. Wang, H. Zhang, K. Li, Z. Lin, J. Yang and X.L. Shene, A hybrid particle swarm optimization algorithm using adaptive learning strategy, *Information Sciences Journal* **436–437** (2018), 162–177.
- [27] A. Lin, W. Sun, H. Yu, G. Wu and H. Tang, Global genetic learning particle swarm optimization with diversity enhancement by ring topology, *Swarm and Evolutionary Computation* **44** (2019), 571–583.
- [28] Y. Marinakis, A. Migdalas and A. Sifaleras, A hybrid Particle Swarm Optimization – Variable Neighborhood Search algorithm for Constrained Shortest Path problems, *European Journal of Operational Research* 2017, pp. 819–834.
- [29] N. Zeng, Z. Wang, W. Liu, H. Zhang, K. Hone and X. Liu, A Dynamic Neighborhood-Based Switching Particle Swarm Optimization Algorithm, in *IEEE Transactions on Cybernetics*, 2020. doi: 10.1109/TCYB.2020.3029748.
- [30] H. Tolabi, A. Lashkar and R. Hosseini, An enhanced particle swarm optimization algorithm to solve probabilistic load flow problem in a micro-grid, *Appl Intell*, 2021, pp. 1645–1668. <https://doi.org/10.1007/s10489-020-01872-4>
- [31] S. Kirkpatrick, Optimization by Simulated Annealing: Quantitative Studies, *Journal of Statistical Physics* **34** (1984), 975–986.
- [32] W. Zhang, G. Wang, Z. Xing and F. Wittenburg, Distributed stochastic search and distributed breakout: properties, comparison and applications to constraint optimization problems in sensor networks, *Artificial Intelligence*, 2005.
- [33] A.R. Jordeh, A review on constraint handling strategies in particle swarm optimisation, *Neural Computing and Applications*, 2015, pp. 1265–1275.
- [34] T. Takahama and S. Sakai, Constrained optimization by the ϵ constrained differential evolution with an archive and gradient-based mutation, *IEEE Congress on Evolutionary Computation*, Barcelona, Spain, 2010.
- [35] T.P. Runarsson and X. Yao, Stochastic ranking for constrained evolutionary optimization, *IEEE Transactions on Evolutionary Computation*, 2000, pp. 284–294.
- [36] P.N. Suganthan, N. Hansen, J.J. Liang, K. Deb, Y.P. Chen, A. Auger and S. Tiwari, Problem Definitions and Evaluation Criteria for the CEC’2005 Special Session on Real-Parameter Optimization, *Technical Report: Nanyang Technological University*, Singapore, 2005.
- [37] X. Li, K. Tang, M.N. Omidvar, Z. Yang and K. Qin, Benchmark Functions for the CEC’2013 Special Session and Competition on Large-Scale Global Optimization, *Technical Report; School of Computer Science and Information Technology*, RMIT University, Melbourne, Australia, 2013.
- [38] J.J. Liang, T. Ph. Runarsson, E.M. Montes, M. Clerc, P.N. Suganthan, C.A. Coello Coello and K. Deb, Problem Definitions and Evaluation Criteria for the CEC 2006: Special Session on Constrained Real-Parameter Optimization, *Technical Report*, 2006.
- [39] H. Abadlia, N. Smairi and Kh. Ghédira, A hybrid Immigrants schema for particle swarm optimization algorithm, Knowledge-Based and Intelligent Information and Engineering Systems: *Proceedings of the 22nd International Conference, Procedia Computer Science*, Elsevier, 2018, pp. 105–115.
- [40] H. Abadlia, N. Smairi and Kh. Ghédira, Particle Swarm Optimization Based on Dynamic Island Model, *29th IEEE International Conference on Tools with Artificial Intelligence, ICTAI*, Boston, MA, USA, 2017, pp. 709–716.
- [41] M.Z. Bigiarini, M. Clerc and R. Rojas, Standard Particle Swarm Optimisation 2011 at CEC-2013: A baseline for future PSO improvements, *IEEE Congress on Evolutionary Computation*, 2013.
- [42] J. Carr, An introduction to genetic algorithms, url: <https://karczmarczyk.users.greyc.fr/TEACH/IAD/GenDoc/carrGenet.pdf>, 2014
- [43] S. Salcedo-Sanz, J. Del Ser, I. Landa-Torres, S. Gil-López and Portilla-Figueras, The coral reefs optimization algorithm: an efficient meta-heuristic for solving hard optimization problems, In: *Proceedings of the 15th International Conference on Applied Stochastic Models and Data Analysis*, 2013, pp. 751–758.
- [44] M.N. Omidvar, M. Yang, Y. Mei, X. Li and X. Yao, DG2: A Faster and More Accurate Differential Grouping for Large-Scale Black-Box Optimization, *IEEE Transaction Evolution Computation*: (2017) p. 929–942.
- [45] R. Cheng and Y. Jin, A social learning particle swarm optimization algorithm for scalable optimization, *Information Sciences*: 2015, pp. 43–60.
- [46] N. Hansen, The CMA evolution strategy: a tutorial, *CoRR* arXiv:1604.00772, 2016.

Copyright of Journal of Intelligent & Fuzzy Systems is the property of IOS Press and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.